Optimizing Claude Code Prompt Caching: Cache Tax and Session Keepwarm Mechanics

Researched 2026-10-12.

1. Reel Claims vs. Verified Reality

In the Instagram Reel (https://www.instagram.com/reel/DeUDrCLhi3H/), creator @aipractically argues that a short prompt sent after a coffee break is the most expensive message in Claude Code, citing a developer who measured a $6.28 cold-rewrite cost on a two-line recap. The reel promotes a free mod called **Cache Tax** that keeps prompt caches warm for ~$0.02 and prevents cold rewrites that allegedly could cost up to $30, while DM-gating installation commands behind social engagement.

What Official Documentation and Repositories Confirm

- **Cache Expiration & Prefix Structure**: According to the official Claude Code Prompt Caching Documentation (https://code.claude.com/docs/en/prompt-caching), prompt caching uses exact server-side prefix matching. Any change to the beginning of a session (or letting the cache expire) causes Claude Code to re-process the entire conversation history. Subscription sessions retain a 1-hour TTL for the main conversation within plan limits, whereas API keys, cloud providers (Bedrock, Google Cloud, Azure Foundry), and usage-credit sessions default to a 5-minute TTL unless explicitly configured with `promptCacheTtl: "1h"`.
- **Cache Tax Tooling**: The karanb192/cache-tax Repository (https://github.com/karanb192/cache-tax) confirms the mod exists (MIT licensed) for Claude Code 2.1.287+. It schedules a single tool-less fork (`$.model.fork`) after 50 idle minutes within an armed window (`/keepwarm`) to refresh the 1-hour cache. The author's measured test on Claude Code 2.1.1 recorded a 330k-token session triggering a $6.61 estimate and a $6.28 actual cache write, while a single keepwarm ping in a 75k-token session read at $0.02.
- **Cold-Send Guard**: The mod implements an execution guard for sessions over 50,000 tokens that drops an initial cold send once, displaying the estimated rewrite price before allowing the user to resend or clear.
- **Subagent Gaps**: As documented in the independent warmer repository izeigerman/claude-thermos (https://github.com/izeigerman/claude-thermos), subagents operate in a separate bucket with a 5-minute TTL. When a main agent blocks waiting for a subagent that exceeds 5 minutes, the main agent's cache expires independently of human coffee breaks. Cache Tax does not protect subagents; `claude-thermos` addresses this via a local proxy wrapper.

What Remains Uncertain or Unverified

- **The $30 Cold Send Claim**: Plausible only as an extrapolation of a ~1.5M token session at Fable 5.1 write rates ($20/MTok), not a measured test documented in the mod's repo.
- **Subscription Quota Deductions**: Anthropic does not publicly document how server-side prompt cache reads impact 5-hour and weekly subscription rate-limit calculations. Claiming that it directly 'drains your limit' in dollar equivalence is an unconfirmed assumption.
- **Fork Cache Bucketing**: Official documentation states forks live in non-main request buckets with standard 5-minute TTLs. While empirical tests by the mod author showed the fork refreshing the main

session's 1-hour countdown on subscription tiers, this behavior is not guaranteed across all providers or configurations.

---

2. Commands & Prerequisites

Prerequisites

- Claude Code CLI version `2.1.287+` (verify via `claude --version`).
- Active Claude Pro/Max subscription (defaults to 1h TTL) OR an API key session with `promptCacheTtl` explicitly set to `"1h"`.
- A terminal supporting status line rendering (e.g., Ghostty, kitty, or standard terminal fallback).

Verified Installation & Usage Commands

Documented in karanb192/cache-tax (https://github.com/karanb192/cache-tax) and karanb192/claude-code-mods (https://github.com/karanb192/claude-code-mods/tree/main/plugins/cache-tax):

1. Add the marketplace registry:
   ```
   claude plugin marketplace add karanb192/claude-code-mods
   ```
2. Install the Cache Tax mod:
   ```
   claude plugin install cache-tax@claude-code-mods
   ```
3. Inside Claude Code, arm the keepwarm timer:
   - `/keepwarm` (Arms default 6-hour window)
   - `/keepwarm 90m` (Arms custom duration)
   - `/keepwarm off` (Disables warming)
   - `/cache-tax` (Inspects cache status card, context size, and tally)

Alternative Subagent Warmer (izeigerman/claude-thermos (https://github.com/izeigerman/claude-thermos)):

```
uvx claude-thermos
```

Note on Reel Commands: The reel creator gatekept the install commands behind DMs, but the actual commands are public, verified marketplace operations.

---

3. Realistic Limits, Risks & Common Mistakes

- **Reach L2 Security Model**: Cache Tax executes inside Claude Code at reach level L2 ('drives Claude'). Per its threat model, it inspects command prefixes, timing, and token counts. Users must evaluate whether their security posture permits third-party mods in un-sandboxed environments.
- **Compaction Breaks Caches**: Running `/compact` replaces history with a new summary prefix. This invalidates the old cache immediately; keepwarm cannot preserve pre-compaction tokens.
- **Prefix Invalidation**: Modifying system instructions, loading new upfront MCP servers, changing model tiers via `/model`, or altering reasoning effort levels invalidates the prefix regardless of whether

keepwarm pinged the server.

- **Hook vs. Mod Duplication**: If migrating from karanb192/claude-code-hooks (https://github.com/karanb192/claude-code-hooks/blob/HEAD/plugins/cache-tax/README.md), remove the hook before installing the mod to prevent duplicate cold-send guards.

---

4. Reusable Verification Prompt

Use this prompt inside Claude Code to evaluate your session's caching profile without risking unauthorized purchases or accidental writes:

```
Inspect the current session's prompt cache metrics and configuration.
1. Check the active Claude Code version and confirm if promptCacheTtl is 1h or 5m.
2. Summarize the context size from the last assistant usage turn (input, cache read, cache write).
3. Calculate the estimated financial cost of a full cold cache rewrite at standard list prices for the current model vs. a single cache read.
4. Check if any MCP servers or recent compaction events have invalidated the cached prefix.
Provide output as a structured diagnostic table without modifying files or running destructive commands.
```

Original sources

- https://www.instagram.com/reel/DeUDrCLhi3H/